

Finding Optimal Encoding Directions via SOS/SDP Certificates

Lester (problem), Claude (approach + implementation)

June 2026

Abstract

This document formalizes Approach 2 for finding the sequence of optimal encoding directions $DD(i)$ for $i = 1, 2, \dots$ in dimension k that maximise the minimum confusable pair size $f(DD(i))$. The method reformulates the problem as an outer optimization over directions, with an inner oracle that certifies lower bounds via the Lasserre moment hierarchy. For $k = 4$ and $s = 2$, this is concretely implementable using exact rational arithmetic and symbolic verification. The approach is rigorous (no tolerances, no measure-zero arguments) but slow.

Contents

1 Problem statement

Given dimension k and number of directions i , we seek to find a set of i encoding directions

$$D^* = (d_1^*, \dots, d_i^*) \in (S^{k-1})^i$$

that maximises the minimum confusable multiset size:

$$f(D^*) = \max_{D \in (S^{k-1})^i} \min \{ s : \text{no confusable pair of size } < s \text{ exists for } D \}.$$

This is an outer optimisation (find best D) with an inner oracle (determine $f(D)$ for fixed D). We cannot compute $f(D)$ exactly in general, but we can compute a ****lower bound**** via the Lasserre certificate.

Remark 1. *We seek $DD(i)$ with $f(DD(i))$ rigorous (certified) — not approximate.*

2 The inner oracle: Lasserre certificates for fixed directions

Setup

Fix encoding directions $D = (d_1, \dots, d_m)$ with $m = i$. The question “does a confusable s -element pair exist?” is equivalent to: does the polynomial system

$$\sum_{v \in X} (v \cdot d_j)^r = \sum_{v \in Y} (v \cdot d_j)^r, \quad j = 1, \dots, m, \quad r = 1, \dots, s, \quad X \neq Y \quad (1)$$

have a solution?

After centring both X and Y at the origin (so $\sum_{v \in X} v = \sum_{v \in Y} v = 0$), this reduces to $(s-1)m$ equations in $2k(s-1)$ variables (the centered coordinates).

The SOS certificate (degree-4 case, $s = 2$)

For $s = 2$, the system reduces to power sums of projections agreeing for $r = 1, 2$. The $r = 1$ condition forces equal centroids (achieved by centring); the $r = 2$ condition is

$$\sum_{v \in X} (v \cdot d_j)^2 = \sum_{v \in Y} (v \cdot d_j)^2 \quad \forall j.$$

Define the polynomial

$$f_D(\vec{A}, \vec{B}) = \sum_{j=1}^m (\vec{A} \cdot d_j)^2 (\vec{B} \cdot d_j)^2, \quad g_D(\vec{A}, \vec{B}) = \|\vec{A}\|^2 \|\vec{B}\|^2,$$

where $\vec{A}, \vec{B} \in \mathbb{R}^k$ represent the two elements of X and Y (centred).

A confusable pair of size 2 exists if and only if $f_D(\vec{A}, \vec{B}) = 0$ for some $(\vec{A}, \vec{B}) \neq (0, 0)$, i.e. if $\min f_D / g_D = 0$.

Definition 1. The $s=2$ SOS certificate for directions D is the maximum real number $\lambda^*(D)$ such that there exist a PSD matrix Q and polynomials p_γ (indexed by monomials γ of degree ≤ 4 in $2k$ variables) satisfying

$$f_D - \lambda^*(D) \cdot g_D = \sum_{\gamma} p_{\gamma}(\vec{A}, \vec{B})^2.$$

Proposition 1. If $\lambda^*(D) > 0$, then $f_D(\vec{A}, \vec{B}) \geq \lambda^*(D) \cdot g_D(\vec{A}, \vec{B})$ for all $(\vec{A}, \vec{B})$. Hence, no non-trivial confusable pair of size 2 exists, and $f(D) \geq 3$.

If $\lambda^*(D) = 0$, the certificate is inconclusive (a confusable size-2 pair may or may not exist).

Computation via SDP

The SOS certificate is computed by solving a semidefinite programme (Section ?? below). The key structure: - **Decision variables**: the Gram matrix Q and the scalar λ - **Objective**: maximise λ - **Constraints**: $Q \succeq 0$ (PSD) and polynomial-identity constraints (linear in Q, λ)

Generalisation to $s \geq 3$ via Lasserre hierarchy

For $s \geq 3$, the SOS approach extends via the Lasserre moment hierarchy (relaxation level d). The moment matrix grows as $\binom{2k+d}{d}$, and higher d gives tighter (larger) lower bounds. For this document we focus on $s = 2$ (a single fixed SDP) and note that $s \geq 3$ requires larger-scale SDPs.

3 SDP formulation for $k=4, s=2$

For concreteness, we work in $k = 4$ (dimension of vectors). The directions d_j are 4-dimensional unit vectors.

Variables and monomials

The $2k=8$ variables are the components of $(\vec{A}, \vec{B})$:

$$A = (A_1, A_2, A_3, A_4), \quad B = (B_1, B_2, B_3, B_4).$$

For degree ≤ 2 monomials in 8 variables, there are $\binom{8+2}{2} = 45$ monomials. The vector $v = (1, A_1, \dots, A_4, B_1, \dots, B_4, A_1^2, A_1 A_2, \dots, B_4^2)$ (all monomials of degree ≤ 2) has length 45.

For degree ≤ 4 monomials, there are $\binom{8+4}{4} = 495$ monomials.

Gram matrix and polynomial identity

We seek a 45×45 PSD matrix Q (the Gram matrix) such that

$$v^T Q v = f_D(A, B) - \lambda \cdot g_D(A, B)$$

as a polynomial identity. Expanding both sides and matching coefficients of each degree-4 monomial γ :

$$\sum_{\substack{\alpha+\beta=\gamma \\ |\alpha|,|\beta|\leq 2}} Q_{\text{idx}(\alpha),\text{idx}(\beta)} = f_{D,\gamma} - \lambda \cdot g_{D,\gamma},$$

where $\text{idx}(\alpha)$ is the index of monomial α in the vector v , and $f_{D,\gamma}, g_{D,\gamma}$ are the known coefficients of f_D and g_D at monomial γ .

The SDP

$$\text{maximise } \lambda \tag{2}$$

$$\text{subject to } Q \succeq 0, \tag{3}$$

$$v^T Q v = f_D - \lambda g_D \quad (\text{polynomial identity}). \tag{4}$$

There are 495 linear constraints (one per degree-4 monomial) and 1 PSD constraint.

Solving with CVXPY + numerical verification

We use CVXPY with a solver (SCS or MOSEK) to compute a numerical solution (Q_*, λ_*) . The solution is approximate (floating-point), but we verify it rigorously (Section ??) before accepting it as a certificate.

4 Rigorous verification without tolerances

After the SDP solver returns a candidate (λ_*, Q_*) , we verify that it is a valid SOS certificate using `**symbolic computation and interval arithmetic**`.

Verification procedure

Input: directions D , candidate (λ_*, Q_*) from SDP solver, dimension $k = 4$.

1. Construct f_D, g_D as symbolic polynomials in SymPy.
2. Construct the polynomial difference $P = f_D - \lambda_* \cdot g_D$ symbolically.
3. Compute the Gram matrix product $v^T Q_* v$ symbolically (as a polynomial in $A_1, \dots, B_4$).
4. Check if $P = v^T Q_* v$ `**exactly**` (as polynomials in SymPy). If yes, return **VERIFIED**.
5. If not exactly equal (likely due to solver error), use interval arithmetic: expand both sides, compare coefficients with interval bounds, check if all intervals contain zero. If yes, the SOS certificate holds approximately and we can tighten λ slightly (via a rounding procedure) to get an exact certificate.
6. If verification fails, return **UNVERIFIED** and discard (λ_*, Q_*) .

Remark 2. *The key advantage: we never compare floating-point numbers directly. We either verify the identity exactly (symbolically) or we use `**rigorous interval arithmetic**` to certify the bounds. No ϵ tolerances on dot products or eigenvalue checks.*

5 Parameterisation of directions

To make the outer search finite, we parameterise unit vectors in $\mathbb{R}^k$ using rational coordinates.

Rational points on the sphere

Definition 2. A rational unit vector in $\mathbb{R}^k$ is a vector $d = (d_1, \dots, d_k)$ where each $d_i \in \mathbb{Q}$ and $\|d\|^2 = 1$ (exactly).

For $k = 4$, we can represent rational unit vectors as:

$$d = \frac{(q_1, q_2, q_3, q_4)}{\sqrt{q_1^2 + q_2^2 + q_3^2 + q_4^2}}, \quad q_i \in \mathbb{Z}.$$

The denominator $\sqrt{\sum q_i^2}$ is an algebraic number (the square root of an integer). Representing and comparing such vectors exactly requires **algebraic arithmetic** (polynomial rings over $\mathbb{Q}$, represented via SymPy’s ‘sqrt’ and ‘Rational’).

Practical strategy: bounded-numerator search

To keep the search finite and tractable:

1. Fix a maximum numerator N (e.g., $N = 20$).
2. Enumerate all integer vectors $(q_1, \dots, q_k)$ with $|q_i| \leq N$, $q_i \neq 0$.
3. Normalize: $d = (q_1, \dots, q_k) / \|q\|$ where $\|q\| = \sqrt{q_1^2 + \dots + q_k^2}$.
4. Store d symbolically as a SymPy vector with algebraic coordinates.
5. For each i -tuple of directions $(d_1, \dots, d_i)$, compute the SDP and verify the certificate.
6. Track the maximum λ^* achieved.

For $k = 4$ and $N = 20$: there are roughly $(2 \cdot 20)^4 / 6 \approx 100,000$ distinct unit directions (accounting for duplicates after normalization). The search over all i -tuples is combinatorially large ($\sim 100,000^i$), but we use heuristics to prune (e.g., only try “interesting” subsets: symmetric configurations, random samples, etc.).

Algebraic vs. floating-point coordinates

When constructing f_D and g_D symbolically, the coefficients will involve square roots (from the normalization). SymPy can handle this, but arithmetic becomes slow.

Alternative (faster): Pre-normalize integer vectors $(q_1, \dots, q_k)$ to float coordinates $d = q / \|q\|_{\text{float}}$, compute the SDP with floats, then verify the result. This is not slower than floating-point but sacrifices the algebraic exactness. It is still rigorous because the **verification** step (Section ??) is symbolic and uses interval arithmetic.

6 Grid search and optimisation

Exhaustive search for small i

For $i \leq 6$: enumerate all i -tuples $(d_1, \dots, d_i)$ of unit directions from the bounded-numerator set, compute $\lambda^*(D)$ for each, return $\arg \max \lambda^*$.

For $i = 7, 8$: the search space is too large ($\sim 10^{28}$); use sampling strategies below.

Sampling strategies for large i

1. ****Symmetric configurations****: regular polytopes (e.g., 24-cell in $\mathbb{R}^4$ has 24 vertices), spherical designs, equiangular tight frames. Compute λ^* for each and track best.
2. ****Random sampling****: draw random i -tuples of unit directions (uniformly on $(S^{k-1})^i$), compute λ^* , track best over many trials.
3. ****Local optimisation****: starting from a promising configuration, use gradient-based methods to improve it. The gradient of $\lambda^*(D)$ w.r.t. D can be computed from the SDP dual solution (envelope theorem).
4. ****Heuristic pruning****: discard i -tuples where some pair of directions is nearly parallel (likely to give small λ^*).

Output and guarantees

For a given i , the output is:

- Directions $DD(i) = (d_1, \dots, d_i)$
- Certified lower bound $\lambda^* > 0$ (if verified)
- Conclusion: $f(DD(i)) \geq 3$ (no size-2 confusable pair).

The true minimum confusable pair size $f(DD(i))$ may be ≥ 3 ; we are only proving a lower bound. To determine if $f(DD(i)) = 3$ exactly, one would need to (a) search for $s = 3$ confusable pairs numerically, and (b) use higher Lasserre levels to tighten the bound.

7 Implementation notes

The proof-of-concept code ‘`confusabledsearch.py`’ implements this approach for $k=4$.

Key functions

1. `sos_sdp_for_directions(D, s=2)`: constructs and solves the SDP for given directions. Returns $(\lambda_*, Q_*, \text{status})$.
2. `verify_sos_certificate(D, lambda_star, Q_star)`: verifies the certificate using SymPy. Returns True/False.
3. `normalize_to_unit_rational(q)`: normalizes an integer vector to a rational unit vector. Returns symbolic SymPy vector.

4. `enumerate_bounded_directions(k, N)`: generates all rational unit vectors with integer numerators $\leq N$. Yields generators (lazy) to avoid memory overload.
5. `search_optimal_directions(k, i, search_type='exhaustive'|'random'|'symmetric')`: searches for the best i -tuple and returns $(DD, \lambda^*, \text{verified})$.

Computational complexity

- For each SDP: $O(495^3)$ (dense matrix operations on 495×495 moment matrix). With a sparse SDP solver, can be reduced. - For each verification: $O(\text{poly}(k, s) \cdot \deg(\text{SymPy algebra}))$. Slow but exact. - Grid size: $O((2N)^{k \cdot i})$ in worst case; reduced by symmetry and sampling.

For $k = 4, i = 4$: exhaustive search over all 4-tuples of directions with $|q_i| \leq 20$ is $\sim 10^{13}$ operations (slow but feasible with optimisation). For $i \geq 7$: use sampling.

Robustness and debugging

- All SymPy expressions are pretty-printed for human inspection. - Failed verifications are logged with the candidate (λ_*, Q_*) for diagnosis. - Interval-arithmetic bounds are tightened iteratively if a certificate is close to verification but fails numerically.

8 Limitations and future work

Known limitations

1. **Limited to $s = 2$** in the current proof-of-concept. Extension to $s \geq 3$ requires Lasserre level $d \geq 2$, which scales poorly in dimension.
2. **Grid search is finite but potentially incomplete**. We may miss the true optimal $DD(i)$ if it lies outside the bounded-numerator set. However, we can increase the bound N to search larger regions.
3. **Verification is slow**. SymPy symbolic computation on large polynomials is expensive. For faster verification, use interval arithmetic alone (which is cheaper but less certain).
4. **No negative results**. If we fail to find a direction set with $\lambda^* > 0$, we cannot conclude that none exists — only that our search was incomplete. To prove non-existence would require exhaustive search or a theoretical argument.

Future extensions

- Lasserre hierarchy for $s \geq 3$ (higher relaxation levels). - Gradient-based outer loop to refine promising direction sets. - Exact algebraic optimization (Gröbner bases) instead of numerical SDP solvers, for complete rigor without floating-point. - Integration with the MinimalConfusableSets hypercube code to compare upper bounds.

9 Relation to other approaches

vs. Approach 1 (exploit $M \vdash k$ structure)

Approach 1 focuses on disproving L-matrix cancellations by hand for special cases. Approach 2 is more systematic: it provides a machine-verifiable lower bound for any direction set, without case analysis.

vs. direct polynomial system solving

One could try to directly find confusable pairs for a given D by solving the polynomial system (??) with Gröbner bases or resultants. This is more general but slower and gives only a yes/no answer, not a certified lower bound. Approach 2 complements this: if the polynomial system has no small solution, the SOS certificate proves it rigorously.

vs. numerical search (`confusable_search.py`)

Numerical search (from the companion document ‘`confusabledp.tex`’) *gives empirical evidence that confusable pairs are certified * evidence via a mathematical proof (the SOS certificate).*